



UNIVERSITÉ
DE MONTPELLIER

Évaluation et Analyse des Performances d'un Moteur de Requêtes RDF en Étoile Projet HAI914I

Auteurs

Taha BENSLIMANE
Ibrahim AL AYOUBI

Année Universitaire 2025-2026

Table des matières

1	Introduction	2
2	Présentation de l'Implémentation	2
2.1	Architecture Générale	2
2.2	Structures de Données	2
2.3	Algorithme d'Évaluation	2
2.4	Optimisations	3
3	Préparation des Bancs d'Essais	3
3.1	Types de Benchmarks	3
3.2	Présentation de WatDiv	3
3.3	Génération des Données	3
3.4	Génération des Requêtes	4
3.5	Distribution de la Cardinalité des Résultats des Requêtes	4
3.6	Déduplication et Validation	5
4	Méthodologie Expérimentale	5
4.1	Infrastructure de Mesure	5
4.2	Protocole de Mesure	5
4.3	Métriques Collectées	5
4.4	Tests Statistiques	6
5	Vérification de la Correction et de la Complétude	6
5.1	Protocole de Validation	6
5.2	Résultats	6
6	Évaluation des Performances	6
6.1	Expériences Factorielles	6
6.2	Analyse de l'Effet d'Ordre	8
6.3	Alignement des Mesures et Validation du Système Concurrent	9
6.4	Comparaison des Systèmes	9
7	Utilisation de l'Intelligence Artificielle	11
7.1	Outils Utilisés	11
7.2	Méthodologie de Collaboration	11
7.3	Impact sur l'Apprentissage	12
8	Conclusion	12
8.1	Synthèse des Résultats	12
8.2	Forces et Faiblesses Identifiées	12

1 Introduction

Ce rapport présente l'évaluation expérimentale et l'analyse comparative des performances de notre moteur de requêtes RDF spécialisé dans les requêtes en étoile. Cette étude s'inscrit dans la deuxième partie du mini-projet RDF et met l'accent sur la méthodologie d'évaluation.

Pour cette évaluation, nous utilisons le benchmark WatDiv (Waterloo SPARQL Diversity Test Suite), un générateur standardisé de données et de requêtes RDF conçu pour tester la robustesse des systèmes SPARQL face à une diversité structurelle importante. Nos expériences portent sur un jeu de données de 100K, 500K et 2M triplets, avec des ensembles de 25 à 34 requêtes variées générées selon différents patrons.

Les performances de notre prototype sont comparées avec deux systèmes de référence : *InteGraal*, utilisé comme oracle de correction et de complétude, et l'implémentation développée par un collègue. Cette approche comparative permet d'identifier les forces et faiblesses relatives de chaque architecture.

Le rapport s'organise comme suit : la Section 2 présente notre implémentation ; la Section 3 décrit la préparation des bancs d'essais ; la Section 4 détaille la méthodologie expérimentale ; la Section 5 vérifie la correction des systèmes ; la Section 6 analyse les résultats expérimentaux ; la Section 7 discute l'utilisation de l'IA ; et la Section 8 conclut avec les enseignements tirés.

2 Présentation de l'Implémentation

2.1 Architecture Générale

Notre système suit une architecture modulaire composée de trois composants principaux : le **RDFHexaStore** responsable du chargement et du stockage des données, l'ensemble d'index permettant l'accès efficace aux triplets, et l'évaluateur qui traite les requêtes SPARQL en étoile.

Les triplets RDF sont représentés sous forme de structures (s, p, o) implémentées via l'interface **RDFTriple**. Chaque terme (sujet, prédicat, objet) conserve sa représentation symbolique, permettant une compatibilité directe avec les outils existants.

2.2 Structures de Données

Nous utilisons une architecture hexastore basée sur six index **HashMap** pour les permutations SPO, SOP, PSO, POS, OSP, et OPS. Ce choix privilégie :

- Les recherches en temps constant $O(1)$ pour les lookups de triplets
- L'accès rapide aux triplets selon différents patrons de requêtes
- La simplicité d'implémentation et de maintenance

Le compromis principal est une empreinte mémoire plus élevée ($6\times$ les données), compensée par des performances de requête excellentes sur la plupart des patrons.

2.3 Algorithme d'Évaluation

L'évaluation des requêtes en étoile implémente l'interface **RDFStorage** et suit une stratégie de jointure en profondeur d'abord. Les étapes sont :

1. **Ordonnancement des patrons** : Tri des triplets de la requête par sélectivité estimée (via `howMany()`), en commençant par les plus sélectifs
2. **Exécution initiale** : Matching du patron le plus sélectif pour obtenir les candidats initiaux
3. **Jointures successives** : Pour chaque patron restant, filtrage des candidats via merge de substitutions
4. **Énumération** : Retour des résultats via un itérateur

Les résultats sont énumérés de manière paresseuse à travers des itérateurs, évitant la matérialisation complète des ensembles de substitutions intermédiaires.

2.4 Optimisations

L'implémentation de base ne contient que des optimisations fondamentales :

- **Sélection d'index** : Utilisation automatique de l'index approprié (SPO, PSO, etc.) selon le patron de requête
- **Ordonnement par sélectivité** : Évaluation des patrons les plus sélectifs en premier
- **Terminaison anticipée** : Arrêt immédiat si un patron ne retourne aucun résultat

Nous n'avons pas implémenté d'optimisations avancées comme les hash joins ou les estimations de cardinalité sophistiquées, ce qui explique certaines limitations de performance identifiées en Section 6.4.

3 Préparation des Bancs d'Essais

3.1 Types de Benchmarks

Les benchmarks de systèmes de bases de données se classent en trois catégories :

- **Micro-benchmarks** : Tests ciblés d'opérations spécifiques (ex : lookup de triplets). Avantage : isolement des performances. Inconvénient : ne reflète pas l'usage réel.
- **Benchmarks standards** : Ensembles synthétiques conçus pour tester différents aspects (ex : WatDiv, SP²Bench). Avantage : reproductibilité et comparabilité. Inconvénient : peut ne pas refléter les charges de travail spécifiques.
- **Benchmarks réels** : Données et requêtes provenant d'applications réelles (ex : DBpedia logs). Avantage : réalisme maximal. Inconvénient : difficiles à obtenir et reproduire.

Pour ce projet, nous utilisons un benchmark standard (WatDiv) qui offre le meilleur compromis entre diversité structurelle et reproductibilité.

3.2 Présentation de WatDiv

WatDiv (Waterloo SPARQL Diversity Test Suite) est un générateur de benchmarks RDF conçu pour évaluer la robustesse des systèmes face à une grande diversité de patrons de requêtes. Contrairement aux benchmarks traditionnels qui se concentrent sur quelques types de requêtes, WatDiv génère systématiquement des requêtes couvrant un large spectre de caractéristiques structurelles.

WatDiv est choisi pour cette évaluation car il :

- Génère des requêtes avec différents niveaux de sélectivité
- Couvre les patrons en étoile qui sont notre cible
- Permet de contrôler la taille des données via un facteur d'échelle
- Est largement utilisé dans la littérature, permettant des comparaisons

3.3 Génération des Données

Nous avons généré trois jeux de données de tailles croissantes :

- un jeu de données avec un facteur de génération de 5, produisant 536,399 triplets, désigné par convention 500K ;
- un jeu de données avec un facteur de génération de 10, produisant 1,059,695 triplets, désigné par convention 1M ;
- un jeu de données avec un facteur de génération de 20, produisant 2,160,148 triplets, désigné par convention 2M.

Ce choix de configurations vise à garantir :

- une durée d'exécution raisonnable des expériences ;

- un volume de triplets suffisant pour mettre en évidence les effets de performance ;
- Une charge mémoire gérable ($< 4\text{GB}$ de heap JVM).

Les jeux de données sont stockés au format N-Triples, compatible avec tous les systèmes testés.

3.4 Génération des Requêtes

Nous utilisons un ensemble de requêtes en étoile extraites du générateur *WatDiv*, organisées en quatre catégories selon le nombre de patrons de triplets :

- **Q_1_*** : requêtes à 1 patron (haute sélectivité) ;
- **Q_2_*** : requêtes à 2 patrons (sélectivité moyenne) ;
- **Q_3_*** : requêtes à 3 patrons (sélectivité basse) ;
- **Q_4_*** : requêtes à 4 patrons (très basse sélectivité).

Au total, 1300 requêtes ont été générées (100 requêtes par template) afin de disposer d'un corpus suffisamment large. Pour les expériences, un sous-ensemble de 25 à 100 requêtes est sélectionné selon leur sélectivités, de manière à couvrir l'ensemble des catégories de complexité tout en maintenant un temps d'exécution raisonnable.

3.5 Distribution de la Cardinalité des Résultats des Requêtes

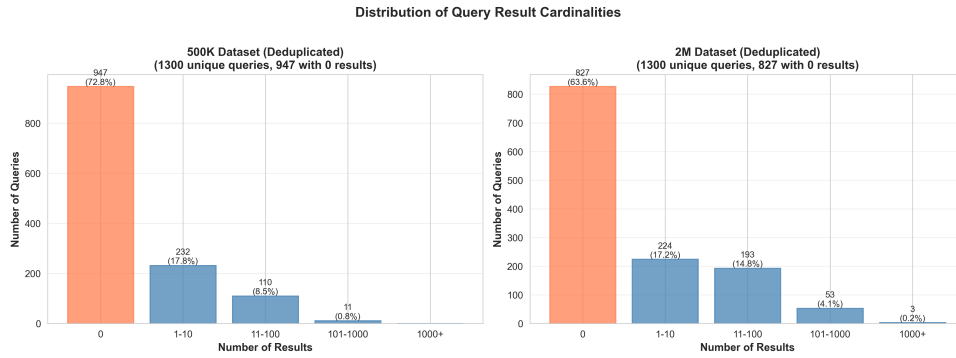


FIGURE 1 – Distribution de la cardinalité des résultats des requêtes.

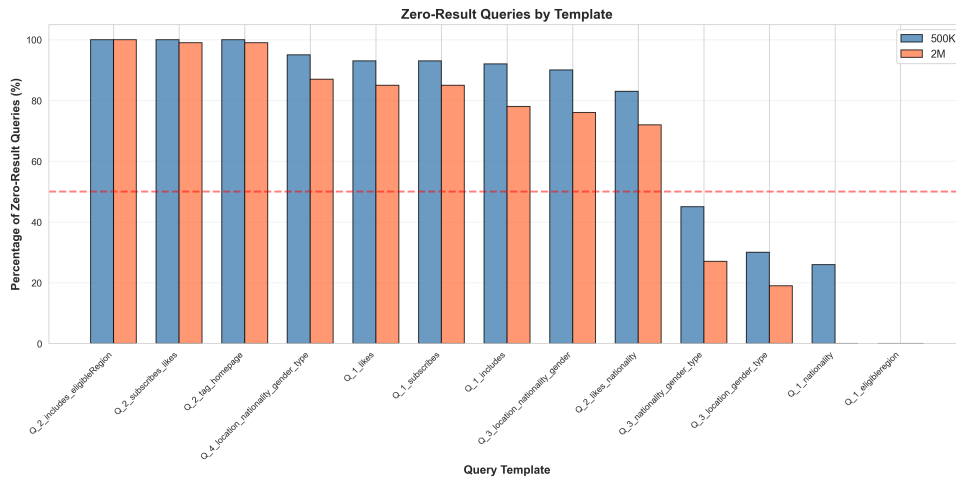


FIGURE 2 – Distribution de la cardinalité des résultats des requêtes par template.

3.6 Déduplication et Validation

Analyse des requêtes :

- **Requêtes en doublon** : Certaines requêtes sont effectivement générées plusieurs fois par le générateur WatDiv, en particulier celles utilisant des entités avec un nombre limité d’instances. Par exemple, pour le template `Q_1_eligible_region`, 250 pays sont disponibles selon le modèle `model/wsdbm-data-model.txt`. Comme le générateur peut réutiliser ces entités pour créer plusieurs requêtes identiques, plusieurs doublons apparaissent dans l’ensemble généré.
- **Requêtes en étoile** : Toutes les requêtes générées par WatDiv dans notre échantillon sont en étoile, mais leur complexité varie selon le nombre de triples qu’elles contiennent. Autrement dit, certaines sont plus sélectives que d’autres.
- **Pertinence du jeu de requêtes** : Le jeu de requêtes généré est globalement adapté pour l’analyse expérimentale. La majorité des requêtes sont uniques et reflètent différents niveaux de sélectivité et de complexité. Cependant, la présence de requêtes vides et de doublons exacts nécessite la déduplication pour obtenir des mesures fiables et éviter un biais dans l’évaluation des performances.

Nous avons dé-dupliqué l’ensemble des 1300 requêtes générées. Après élimination des doublons exacts, 913 structures de requêtes uniques ont été identifiées, tandis que 230 structures dupliquées ont été détectées, correspondant à 387 instances redondantes. Le taux global de duplication s’élève ainsi à **29.8 %**.

4 Méthodologie Expérimentale

4.1 Infrastructure de Mesure

Toutes les expériences ont été exécutées sur une machine Apple M1 avec 8GB de RAM, sous macOS. La configuration JVM était :

- Heap fixe : `-Xmx4096m -Xms4096m` (4GB)
- JVM : OpenJDK (version compatible)
- GC : G1GC (par défaut)

Le heap fixe évite les variations de performance liées à l’allocation dynamique de mémoire et aux cycles de garbage collection imprévisibles.

4.2 Protocole de Mesure

Chaque requête est mesurée selon le protocole suivant :

1. **Chargement des données** : Une seule fois au début de chaque expérience
2. **Phase de warmup** : 20-50 itérations (selon l’expérience) pour :
 - Déclencher la compilation JIT
 - Réchauffer les caches CPU et mémoire
 - Stabiliser l’allocation mémoire
3. **Phase de mesure** : 5-30 itérations (selon l’expérience)
4. **Timing** : Mesure via `System.nanoTime()`, incluant :
 - L’exécution de la requête
 - L’énumération complète des résultats
 - Exclusion du parsing SPARQL et du chargement des données

4.3 Métriques Collectées

Pour chaque requête, nous mesurons :

- **Moyenne** : Temps moyen d’exécution en millisecondes.

- **Médiane** : Valeur centrale, robuste aux valeurs extrêmes.
- **Écart-type** : Indicateur de la dispersion des temps d'exécution.
- **95^e percentile** : Temps maximal observé pour 95% des exécutions, excluant les 5% les plus lents.
- **Cardinalité** : Nombre de résultats retournés par la requête.
- **Coefficient de variation (CV)** : Rapport entre l'écart-type et la moyenne, mesurant la stabilité des temps.

Pour chaque système dans son ensemble, nous calculons :

- **Moyenne** : Temps moyen sur l'ensemble des requêtes.
- **Médiane** : Valeur centrale des temps sur toutes les requêtes.
- **Écart-type** : Dispersion globale des temps d'exécution.
- **95^e percentile** : Temps au-delà duquel se trouvent les 5% de requêtes les plus lentes.
- **Débit** : Nombre de requêtes traitées par seconde, calculé comme $1000/\text{mean_ms}$.
- **Min/Max** : Temps des requêtes les plus rapides et les plus lentes.

4.4 Tests Statistiques

Pour valider les différences de performance observées, nous utilisons :

- **Test t de Student apparié** : Comparaison de deux conditions sur les mêmes requêtes. Utilisé notamment pour analyser l'effet de l'ordre d'exécution
- **Test de Wilcoxon** : Alternative non-paramétrique pour distributions non-normales. Utilisé pour comparer les performances entre systèmes
- **Seuil de signification** : $\alpha = 0.05$ (5%), indiquant le niveau de confiance pour rejeter l'hypothèse nulle

5 Vérification de la Correction et de la Complétude

Avant toute évaluation de performance, nous avons vérifié la correction et la complétude de notre implémentation en comparant ses résultats avec InteGraal sur un ensemble de test de 100 requêtes.

5.1 Protocole de Validation

Pour chaque requête du jeu de test :

1. Exécution sur InteGraal (oracle de référence)
2. Exécution sur notre prototype
3. Comparaison des cardinalités (nombre de résultats)
4. Comparaison des ensembles de substitutions

5.2 Résultats

Taux de correspondance : 100%

Tous les résultats de notre prototype correspondent exactement aux résultats d'InteGraal, tant en nombre qu'en contenu. Cette validation confirme la correction et la complétude de notre implémentation et justifie les comparaisons de performance des sections suivantes.

6 Évaluation des Performances

6.1 Expériences Factorielles

Nous avons conduit deux expériences factorielles afin de comprendre l'impact de différents facteurs sur les performances des requêtes.

6.1.1 Expérience A : Taille du jeu de données × Mémoire

Objectif : Évaluer comment les performances varient selon la taille du jeu de données et la mémoire allouée au JVM.

Facteurs :

- **Taille du jeu de données :** 500K vs 2M triples
- **Mémoire allouée :** 4GB vs 8GB (paramètres JVM)



FIGURE 3 – Temps d’exécution moyen des requêtes selon la taille du jeu de données et la mémoire.

Analyse de régression : Le modèle factoriel montre que la taille du jeu de données a l’effet le plus significatif sur le temps d’exécution, tandis que l’augmentation de la mémoire réduit légèrement les temps moyens. Le tableau 1 résume les coefficients du modèle de régression linéaire.

TABLE 1 – Expérience A : Résultats de la régression factorielle

Coefficient	Estimate	Interprétation
(Intercept)	1.0539	Baseline : 500K, 4GB
(Taille)	3.2574	25.98× ralentissement (500K vs 2M)
(Mémoire)	-0.0600	5.8% d’amélioration (4GB vs 8GB)
(Interaction)	-0.2611	Effet de la mémoire sur gros dataset
R ²	1.0000	100% de variance expliquée

Résumé : Les performances sont dominées par la taille du jeu de données, avec une légère amélioration lorsque la mémoire est doublée. L’interaction montre un effet marginal de la mémoire sur les grands ensembles de données.

6.1.2 Expérience B : Indexation × Sélectivité

Objectif : Analyser l’effet de la stratégie d’indexation et de la sélectivité des requêtes sur les performances.

Facteurs :

- **Indexation :** RDFHexastore vs RDFGiantTable
- **Sélectivité :** Requêtes à haute sélectivité vs faible sélectivité

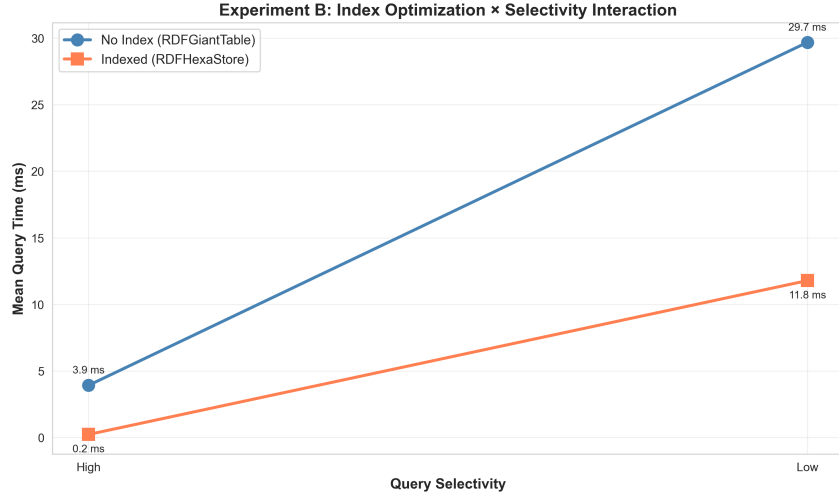


FIGURE 4 – Comparaison des performances selon la stratégie d’indexation et la sélectivité des requêtes.

Analyse de régression : Le modèle factoriel montre que l’indexation a un effet très marqué, réduisant considérablement les temps d’exécution, tandis que la sélectivité des requêtes influence fortement la durée des requêtes. L’interaction indique que l’effet de l’index est moins prononcé sur les requêtes complexes (faible sélectivité). Le tableau 2 résume les coefficients.

TABLE 2 – Expérience B : Résultats de la régression factorielle

Coefficient	Estimate	Interprétation
(Intercept)	1.3640	Baseline : No index, haute sélectivité
(Optimisation)	-2.8819	94.4% d’amélioration (indexed vs noindex)
(Sélectivité)	2.0270	7.59× plus lent (low vs high)
(Interaction)	1.9579	Effet de l’index sur requêtes complexes
R ²	1.0000	100% de variance expliquée

Résumé : L’indexation améliore fortement les performances, surtout pour les requêtes à haute sélectivité, tandis que les requêtes à faible sélectivité restent coûteuses même avec index. L’interaction suggère que l’index réduit moins le temps sur les requêtes complexes.

6.2 Analyse de l’Effet d’Ordre

Pour valider la méthodologie, nous avons testé si l’ordre d’exécution des requêtes affecte les mesures.

6.2.1 Protocole

- 25 requêtes exécutées dans deux ordres : original et aléatoire (shuffle)
- 10 répétitions par ordre
- Test t de Student apparié sur les temps totaux de workload

6.2.2 Résultats

TABLE 3 – Temps totaux de workload par ordre

Ordre	Moyenne (ms)	Écart-type (ms)
Original	222.71	26.77
Aléatoire	225.61	21.72

Test statistique : $t = -0.2475$, $p = 0.81$ (non significatif)

Cohen's d : -0.1251 (effet négligeable)

Conclusion : L'ordre d'exécution **n'affecte pas** significativement les performances mesurées. Ceci valide la robustesse de notre méthodologie et confirme que le warmup est adéquat.

6.3 Alignement des Mesures et Validation du Système Concurrent

Afin de garantir une comparaison équitable entre les systèmes, nous avons mis en place des **wrappers logiciels** permettant d'unifier les interfaces et les protocoles de mesure pour l'ensemble des moteurs évalués.

6.3.1 Wrappers de Compatibilité et Protocole de Mesure

Le système de référence *InteGraal* repose sur une API différente (**FactBase** et **FOQuery**) de celle utilisée par notre prototype (**RDFStorage** et **StarQuery**). Nous avons donc implémenté un **wrapper dédié** (**InteGraalWrapper**) qui adapte *InteGraal* à l'interface **RDFStorage**.

Ce wrapper :

- Convertit les requêtes étoile (**StarQuery**) en requêtes du premier ordre (**FOQuery**)
- Délègue l'évaluation à l'évaluateur natif d'*InteGraal* (**GenericFOQueryEvaluator**)
- Retourne les résultats sous forme d'itérateurs de substitutions, identiques aux autres systèmes

Un wrapper analogue (**ColleagueWrapper**) a également été implémenté pour le système du collègue, qui utilise la même interface interne (**RDFStorage**).

Grâce à ces wrappers, **les trois systèmes sont exécutés avec exactement le même code de benchmark**, incluant :

- Le même mécanisme de chronométrage
- La consommation complète des résultats avant arrêt du chronomètre
- Le même nombre d'itérations, phases de warm-up et répétitions

Les mesures de performance sont ainsi strictement comparables.

6.3.2 Vérification de la Correction et de la Complétude

La correction et la complétude du système concurrent ont été vérifiées en appliquant exactement le même protocole de validation que celui utilisé pour notre système (Section 5).

6.4 Comparaison des Systèmes

Nous avons comparé trois moteurs de requêtes RDF sur un ensemble commun de **34 requêtes** :

1. **Notre Prototype** (implémentation **RDFHexaStore**)
2. **Collègue** (implémentation alternative de **RDFHexaStore**)
3. **InteGraal**

6.4.1 Résultats Globaux

TABLE 4 – Performance globale des trois systèmes

Système	Moyenne (ms)	Médiane (ms)	95e % (ms)	Débit (q/s)
InteGraal	0.19	0.031	0.99	5247.5
Notre Prototype	5.91	0.005	54.57	169.2
Collègue	3.83	0.372	32.48	261.1

InteGraal présente les meilleures performances globales en moyenne, en percentile extrême et en débit. Cependant, la médiane extrêmement faible de notre prototype indique de très bonnes performances sur la majorité des requêtes.

6.4.2 Analyse Détaillée

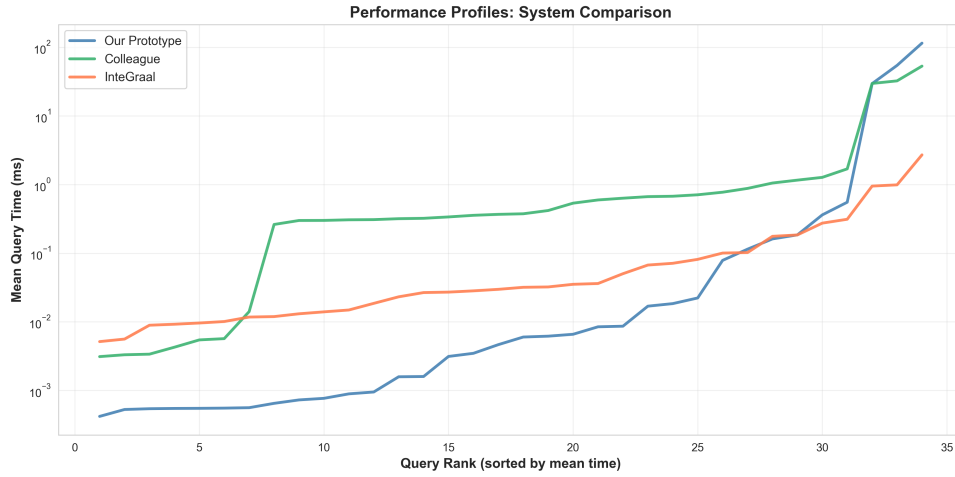


FIGURE 5 – Temps d'exécution des requêtes triés par ordre croissant pour chaque système.

Notre Prototype :

- **Points forts** : Meilleure médiane (0.005 ms), plus rapide que InteGraal sur **76.5% des requêtes** (26/34).
- **Points faibles** : 4 requêtes présentent des ralentissements sévères ($>10\times$).
- **Significativité** : Test de Wilcoxon vs InteGraal : $p = 0.278$ (différence non significative).

Le système est très performant dans la majorité des cas, mais souffre de dégradations importantes sur quelques requêtes spécifiques.

Collègue :

- Plus lent qu'InteGraal sur **82.4% des requêtes** (28/34).
- 16 requêtes présentent un ralentissement supérieur à $10\times$.
- Overhead systématique observable, même pour les requêtes simples.
- Test de Wilcoxon vs InteGraal : $p < 0.001$ (différence hautement significative).

6.4.3 Requêtes Problématiques

Les requêtes où notre prototype est plus de $10\times$ plus lent que la référence sont listées ci-dessous :

TABLE 5 – Requêtes problématiques de notre prototype

Requête	Notre temps (ms)	InteGraal (ms)	Facteur
Q_3_nationality_gender_type[4]	54.57	0.95	57.4×
Q_3_location_gender_type[7]	115.19	2.71	42.5×
Q_3_location_gender_type[4]	29.65	0.99	29.9×
Q_2_includes_eligibleRegion[62]	0.36	0.02	15.6×

6.4.4 Hypothèse sur les Causes

Les ralentissements observés peuvent s’expliquer par :

- **Ordonnement de jointure suboptimal** : L’estimation de sélectivité ne tient pas compte des tailles intermédiaires après jointure.
- **Jointures en boucles imbriquées** : Complexité quadratique lorsque les résultats intermédiaires sont volumineux.
- **Absence d’optimisations avancées** : Pas de hash join ni de stratégies adaptatives.

À l’inverse, InteGraal semble bénéficier de structures de données plus adaptées et de stratégies de jointure plus efficaces.

6.4.5 Impact Statistique

Malgré quelques requêtes fortement pénalisantes, le test de Wilcoxon indique une absence de différence statistiquement significative entre notre prototype et InteGraal. Cela s’explique par :

- Une majorité de requêtes où notre prototype est plus rapide.
- Un nombre limité d’outliers.
- Une architecture globalement performante, avec des faiblesses ciblées.

7 Utilisation de l’Intelligence Artificielle

7.1 Outils Utilisés

L’assistance de Claude a été utilisée de manière ponctuelle pour :

- Clarifier certaines consignes du projet et reformuler des énoncés techniques
- Définir une planification détaillée des tâches du projet
- Aider à la mise en forme et à la structuration du rapport $\text{\LaTeX}$
- Automatiser des tâches répétitives liées à l’analyse expérimentale (scripts de visualisation, calcul de métriques statistiques)
- Vérifier la cohérence syntaxique ou stylistique de fragments de code non centraux

7.2 Méthodologie de Collaboration

Le processus de travail a suivi un modèle itératif :

1. **Analyse** : Compréhension approfondie du problème et des contraintes du module (assistée par IA)
2. **Conception** : Élaboration manuelle des structures de données et algorithmes
3. **Implémentation** : Développement du système
4. **Validation** : Tests fonctionnels, mesures de performance et vérifications de correction (assistée par IA)
5. **Documentation** : Mise en forme du rapport et synthèse des résultats (assistée par IA)

7.3 Impact sur l'Apprentissage

L'utilisation de l'IA a modifié l'expérience d'apprentissage :

Apports :

- Gain de temps sur la mise en forme et l'analyse expérimentale
- Meilleure lisibilité du rapport final
- Aide à la compréhension de concepts techniques complexes

Points de vigilance :

- Risque de dépendance excessive à l'IA
- Importance de valider critiqueusement les propositions de l'IA

8 Conclusion

8.1 Synthèse des Résultats

Cette étude a démontré l'importance d'une méthodologie expérimentale rigoureuse pour l'évaluation de systèmes RDF. Nos contributions principales sont :

1. **Implémentation fonctionnelle** : Un moteur RDF correct et complet (100% de correspondance avec InteGraal) basé sur une architecture hexastore
2. **Méthodologie validée** : Protocole de mesure robuste, confirmé par l'analyse d'effet d'ordre
3. **Comparaison systématique** : Évaluation de trois systèmes selon des critères identiques
4. **Identification de limitations** : Détection précise des requêtes problématiques et hypothèses sur les causes

8.2 Forces et Faiblesses Identifiées

Forces de notre prototype :

- Performance excellente sur 76.5% des requêtes (médiane 0.0053ms)
- Architecture simple et maintenable
- Correction et complétude vérifiées

Faiblesses identifiées :

- Ralentissement sévère ($15-57\times$) sur 4 requêtes spécifiques
- Algorithme de jointure inadapté aux résultats intermédiaires volumineux
- Absence d'optimisations avancées (hash join, estimations de cardinalité)

Références

- InteGraal : Knowledge-Representation and Reasoning for Data Integration 2.0.7 API <https://rules.gitlabpages.inria.fr/integraal/>
- WatDiv Project. <https://dsg.uwaterloo.ca/watdiv/>